



opensophy
13 мая в 11:57

Linux: Права доступа



Средний



12 мин



15K

Учебный процесс в IT, Терминология IT, Инженерные системы*, Информационная безопасность*, Настройка Linux

Тutorial

Введение

Продолжаем серию рубрики Linux. В прошлой серии разобрали пользователей и группы – UID, GID, `/etc/passwd`. Теперь разберём что ядро делает с этими числами когда процесс пытается открыть файл или войти в директорию. Про права доступа легко выучить команды – `chmod 755`, `chown user file` – но не понимать что происходит под капотом. Тогда начинаются странные ситуации: «я же владелец, почему `Permission denied`?». Цель этой статьи – кратко но в достаточной форме рассказать про права доступа.

Вся статья построена на практике: каждый раздел можно пройти руками. Для большинства примеров достаточно обычного пользователя, для части нужен `sudo`. Где нужен – отмечено явно. Серия в основном под Ubuntu/Debian.

В конце статьи будет бонус который даст вам возможность изучить права доступа в Linux ещё доступнее и возможно проще.

Обновление статьи: 2026-05-14

1. Что такое права доступа и зачем они нужны

Linux – многопользовательская система. На одной машине одновременно работают процессы от разных пользователей: `nginx` читает файлы сайта, `postgres` работает со своими данными, разработчик `ivan` пишет код в своём каталоге. Без изоляции любой мог бы читать чужие ключи SSH, удалять чужие файлы, подменять конфиги сервисов.

Права доступа – механизм изоляции. Каждый файловый объект несёт метаданные: кто владелец, какая группа, кто что может делать. Метаданные хранятся в **inode** – структуре файловой системы которая описывает файл. Важно понимать: содержимое файла и его метаданные (включая права) хранятся раздельно. Когда вы делаете `ls -la` – вы читаете именно inode, а не сам файл.

Посмотрим на конкретный пример. Файл `/etc/shadow` хранит хеши паролей всех пользователей:

РЕКЛАМА



Слишком умный?
Пройди тест



```
-rw-r----- 1 root shadow 1478 apr 16 12:03 /etc/shadow
```

Объяснить с 

Права гарантируют что читать его может только root и члены группы shadow . Обычный пользователь – нет. Разберём эту строку полностью, слева направо.

Часть	Что означает
-rw-r-----	тип файла и биты прав (разберём ниже)
1	количество жёстких ссылок на файл
root	владелец файла
shadow	группа-владелец файла
1478	размер в байтах
apr 16 12:03	дата последнего изменения
/etc/shadow	путь к файлу

2. Модель UGO – три класса доступа

Стандартная модель Linux делит всех на три класса относительно конкретного файла.

U – User (владелец). Один конкретный пользователь. Когда создаёте файл – автоматически становитесь его владельцем. **G – Group (группа).** Одна конкретная группа. По умолчанию – primary group пользователя который создал файл. **O – Others (остальные).** Все кто не попал в первые два класса.

Для каждого из трёх классов устанавливается свой набор прав. Вот как это выглядит в выводе `ls -la`:

```
veil@veilos:~$ ls -la
drwxr-x---+ 19 veil veil 4096 apr 24 13:14 .
```

Объяснить с 

Первый символ – тип файла:

Символ	Тип
-	обычный файл
d	директория
l	символическая ссылка
b	блочное устройство
c	символьное устройство
p	именованный канал (pipe)



Слишком умный?
Пройди тест

```
- rw- r-- ---
U   G   O
```

U=rw- → владелец (root) читает и пишет
G=r-- → группа (shadow) только читает
O=--- → все остальные – никаких прав

Объяснить с 

Каждый файл имеет ровно одного владельца и ровно одну группу – не двух, не список. Это ограничение стандартной модели UGO. Для более гибкого управления существуют ACL – разберём их в разделе 9.

3. Биты gwx – что они означают

Три бита в каждом блоке: **r** – read, **w** – write, **x** – execute. Установлен бит – операция разрешена. Сброшен (дефис) – запрещена. Смысл битов различается для файлов и директорий.

Для файлов

Бит	Что разрешает
r	читать содержимое (cat , less , cp)
w	изменять содержимое (редакторы, echo >>)
x	запускать как программу

Для директорий

Здесь важно не путать – директория это тоже файл, просто специального типа. Биты работают иначе.

Бит	Что разрешает
r	читать список файлов внутри (ls)
w	создавать и удалять файлы внутри
x	входить (cd) и обращаться к файлам по имени

На директориях бит **x** (проход) нужен, чтобы в дальнейшем войти через **cd** или обратиться к файлу по пути. **r** без **x** будет парадоксом, где список файлов через **ls** будет видно а открыть нельзя, поэтому **r** и **x** на директориях почти во многих случаях ставятся вместе.

4. Числовое и символьное представление

Существует два способа записывать права, и оба используются в реальной работе. Разберём оба.

Числовое (восьмеричное)

Каждый бит имеет числовое значение:



Слишком умный?

Пройди тест

1 = 4
w = 2

```
x = 1
- = 0
```

Объяснить с 

Чтобы получить число для класса – складываем биты:

```
rwX = 4+2+1 = 7
rw- = 4+2+0 = 6
r-X = 4+0+1 = 5
r-- = 4+0+0 = 4
--- = 0+0+0 = 0
```

Объяснить с 

Итоговые права – три цифры, по одной для U, G, O:

```
755 → rwX r-X r-X
644 → rw- r-- r--
700 → rwX --- ---
600 → rw- --- ---
```

Объяснить с 

777 означает что любой пользователь на системе может читать, изменять и запускать файл. На сервере где работают несколько пользователей – это дыра. Особенно опасно если это скрипт который запускается по cron от root – любой сможет подменить его содержимое. Используйте с осторожностью.

Шпаргалка по типовым комбинациям:

Права	Символьно	Когда применять
644	rw-r--r--	Обычные файлы, конфиги
755	rwXr-Xr-X	Исполняемые файлы, директории
600	rw-----	Приватные ключи SSH, секреты
640	rw-r-----	Конфиги с паролями – группа читает, others нет
700	rwX-----	Домашняя директория сервисного пользователя
750	rwXr-X---	Директории сервисов
777	rwXrwXrwX	Почти никогда – проблема безопасности

Символьное

Удобно когда нужно добавить или убрать один бит не трогая остальные – не надо держать в голове итоговое число. Синтаксис: [кто] [операция] [что] .

Кто: u – владелец, g – группа, o – остальные, a – все сразу (u+g+o).

Операция: + – добавить, - – убрать, = – установить точно (остальные биты сбросятся).



Слишком умный?

Пройди тест

```
chmod g-w file.txt      # убрать w у группы
chmod o=r file.txt      # ровно r для others, остальное сбрасывается
chmod a+x file.sh       # добавить x всем
```

```
chmod u+x,g-w file.txt    # несколько операций через запятую
chmod go-rwx secret.txt   # убрать всё у группы и others
```

Объяснить с 

Новый навык за несколько недель

когда хочешь изменить один бит не затрагивая остальные.

Особый случай: +X (заглавная)

+x (строчная) ставит бит execute на всё подряд – и на файлы, и на директории. +X (заглавная) её можно назвать умнее: она ставит execute только там, где это имеет смысл – на директории всегда, на файлы только если у кого-то из классов уже стоит x .

```
chmod -R +x dir/          # x поставится на ВСЕ файлы внутри, включая .txt, .jpg
chmod -R +X dir/          # x поставится только на директории и уже исполняемые файлы
```

Объяснить с 

Пример где можно применить: для рекурсивной установки прав.(Это безопаснее и правильнее чем потом чистить лишние биты через find .)

5. chmod – изменение прав

chmod (от *change mode*) – команда для изменения прав доступа.

Числовым способом



+27



139



8

```
chmod 600 ~/.ssh/id_rsa
```

Символьным способом

```
chmod u+x script.sh
chmod g+w shared/
chmod o-rwx secret.txt
```

Рекурсивно для всего дерева каталогов

```
chmod -R 750 /var/lib/app
```

Объяснить с 

Здесь есть типичная ловушка при настройке веб-серверов. Многие делают так:

```
chmod -R 755 /var/www/html
```

Объяснить с 

Эта команда поставит 755 на абсолютно все объекты – и директории, и файлы. Для директорий 755 нормально, им нужен x чтобы в них можно было входить. Но для файлов это означает что .html , .css , .jpg становятся исполняемыми – а этого не должно быть. Правильный способ – разделить файлы и директории:

Директориям нужен x чтобы в них можно было входить



Слишком умный?

Пройди тест

```
find /var/www/html -type f -exec chmod 644 {} \;
```

Объяснить с 

После этого права будут корректные: директории – 755 , файлы – 644 , без лишнего x .
Главное: не ставить права «на глаз» – всегда думайте кому реально нужен доступ.

6. chown и chgrp – смена владельца и группы

chown (от *change owner*) меняет владельца файла, группу, или оба сразу. chgrp (от *change group*) меняет только группу – по сути то же что chown :group , просто отдельная команда.

```
# Сменить только владельца
chown ivan file.txt

# Сменить только группу
chgrp developers file.txt

# Сменить и владельца и группу сразу
chown ivan:developers file.txt

# Рекурсивно
chown -R www-data:www-data /var/www/html
```

[Объяснить с](#) 

Обычный пользователь может поменять группу файла только на одну из своих собственных групп. Поменять владельца на другого пользователя – только root. Без этого ограничения можно было бы «подарить» вредоносный файл с SUID другому пользователю – и это стало бы простым способом эскалации привилегий.

7. Как ядро принимает решение о доступе

Это самый важный раздел статьи. Команды можно выучить, но без понимания алгоритма поведение системы будет казаться непредсказуемым. Особенно когда видишь Permission denied там где не ожидаешь.

7.1 Real vs Effective UID

Когда пользователь запускает программу, у процесса есть несколько идентификаторов. **RUID (Real UID)** – кто фактически запустил процесс. Не меняется в течение жизни процесса. **EUID (Effective UID)** – от чьего имени процесс работает прямо сейчас. Именно его проверяет ядро при обращении к файлу.

В большинстве случаев RUID == EUID. Но это различие становится важным при SUID – разберём в следующем разделе.

```
cat /proc/self/status | grep Uid
```

[Объяснить с](#) 

```
Uid:    1000    1000    1000    1000
```

[Объяснить с](#) 

Четыре числа в строке: Real UID, Effective UID, Saved UID (нужен для временного переключения прав), Filesystem UID (используется при проверке доступа к файлам – в большинстве случаев совпадает с EUID). Для понимания прав доступа ключевой – второй,

**Слишком умный?**

Пройди тест

Когда процесс обращается к файлу, ядро проверяет права строго по порядку и останавливается на первом совпадении:

1. Если EUID = 0 (root) → доступ почти всегда разрешён
2. Если EUID совпадает с владельцем файла → используются права владельца
3. Если группа процесса (EGID или дополнительные группы) совпадает с группой файла → используются права группы
4. Во всех остальных случаях → используются права для остальных (others)

Система не комбинирует права – она берёт только один подходящий вариант и на этом заканчивает проверку.

7.3 Права не суммируются

Частая ловушка с которой сталкиваются многие. Допустим есть файл:

```
ls -la secret.txt
# ---rwx--- 1 ivan developers secret.txt
# U: ---   G: rwx   O: ---
```

Объяснить с 

Пользователь `ivan` является владельцем и одновременно входит в группу `developers` :

```
id ivan
# uid=1001(ivan) gid=1001(ivan) groups=1001(ivan),1005(developers)
```

Объяснить с 

Но если `ivan` попытается прочитать файл – получит `Permission denied`. Почему? Ядро проверило: EUID процесса совпадает с UID файла – совпадение на шаге 2. Применило биты `owner` – `---`. Остановилось. Биты `group` `rwx` даже не рассматривались. `ivan` является владельцем, но у владельца нет прав. Группа имеет полный доступ – но до неё проверка не дошла.

Решение – дать владельцу права явно:

```
chmod u+r secret.txt
# или явно задать нужные права
chmod 740 secret.txt # u=rwx, g=r--, o=---
```

Объяснить с 

Уточнение: пример выше демонстрирует механику проверки прав ядром, а не реальную схему работы с группами. Ситуация `----rwx---` на файле, где владелец намеренно без прав, надуманная и в реальной практике почти не встречается.

Зачем тогда нужны группы?

Главная задача групп – дать одинаковые права произвольному набору пользователей без того, чтобы менять права на каждом файле вручную. Это работает именно потому что проверка `group`-битов срабатывает когда пользователь **не является владельцем** файла.

Примерный сценарий: команда разработчиков работает с общим проектом.



Слишком умный?

Пройди тест

```
groupadd developers
usermod -aG developers maria
usermod -aG developers sergei
```

```
# Назначить группу на директорию проекта
chown -R ivan:developers /project
chmod -R 750 /project
```

Объяснить с 

Теперь `ivan` – владелец (`rwX`), все члены группы `developers` – читают и входят (`r-X`), остальные – ничего. Когда нужно добавить нового разработчика – просто `usermod -aG developers newuser`, и он сразу получает нужный доступ. Группы становятся особенно полезными в связке с SGID на директории (раздел 8): новые файлы автоматически наследуют группу директории, и всем членам группы не нужно думать о правах при создании файлов.

7.4 root и права

`root` (EUID=0) обходит проверку UGO для большинства операций – может читать любой файл, писать в любой файл независимо от прав. Но есть одно исключение: бит `execute`. Даже `root` не может запустить файл если ни у кого не установлен `X`:

```
chmod 644 script.sh
sudo ./script.sh
# ./script.sh: Permission denied – даже для root
```

Объяснить с 

Чтобы запустить – сначала `chmod +X script.sh`, потом `sudo ./script.sh`. Это защита от случайного запуска: если `X` не установлен, файл не является исполняемым ни для кого, включая `root`.

8. Специальные биты – SUID, SGID, Sticky bit

Помимо стандартных `rwX` существуют три специальных бита. Каждый работает по-разному в зависимости от того – установлен он на файл или на директорию.

SUID (Set User ID)

При запуске файла с SUID процесс получает **EUID = UID владельца файла**, а не UID пользователя который его запустил. Пример – команда `passwd`. Обычный пользователь должен иметь возможность сменить свой пароль, а значит записать новый хеш в `/etc/shadow`. Но `/etc/shadow` доступен только `root`. SUID решает эту проблему: `/usr/bin/passwd` принадлежит `root` и имеет SUID – при запуске процесс временно получает EUID=0 и может писать в `shadow`.

```
ls -la /usr/bin/passwd
# -rwsr-xr-x 1 root root 64152 Feb 6 /usr/bin/passwd
```

Объяснить с 

`s` вместо `X` в блоке владельца – SUID установлен. Заглавная `S` (без `X`) означает что SUID установлен но бит `execute` при этом не установлен – это подозрительная ситуация которой обычно не должно быть.

```
# Установить SUID
chmod u+s file
```



Слишком умный?

Пройди тест

```
find / -perm -4000 -type f 2>/dev/null
```

Объяснить с 

SUID-бинарники – популярная цель при атаках на повышение привилегий. Если на стандартной утилите вроде `vim`, `find` или `bash` внезапно появился SUID – это серьёзный повод для расследования. Регулярно проверяйте список SUID-файлов и сравнивайте с эталоном.

Эталон – это список SUID-файлов, снятый в заведомо чистом состоянии системы (сразу после установки или после проверенного аудита). При следующих проверках вы сравниваете текущий список с ним и видите, что появилось нового.

SGID (Set Group ID)

SGID работает по-разному для файлов и директорий. **На файл:** процесс получает **EGID = GID группы файла** – аналог SUID но для группы. Используется редко. **На директорию:** все файлы и поддиректории созданные внутри автоматически наследуют группу этой директории, а не primary group создателя. Это ключевой механизм для совместной работы. В качестве примера возьмём несколько разработчиков которые работают с одной директорией. Без SGID каждый создаёт файлы со своей primary group – другие не могут их редактировать даже если входят в нужную группу. С SGID все файлы автоматически получают группу директории и все члены группы могут работать с ними без лишних телодвижений.

```
# Установить SGID на директорию
chmod g+s /project
chmod 2755 /project    # 2 – это SGID в числовом виде

# Проверить – в выводе ls будет s вместо x в блоке группы
ls -la /project
# drwxr-sr-x 2 ivan developers 4096 apr 29 /project
```

Объяснить с 

Sticky bit

Sticky bit решает конкретную проблему: если у пользователя есть право `w` на директорию – он может удалить из неё любой файл, даже чужой. **На директорию:** пользователь может удалить файл только если он является владельцем файла или владельцем директории. Просто иметь `w` на директорию – недостаточно. Пример – `/tmp`, где директория доступна на запись всем, но никто не должен удалять чужие файлы:

```
ls -la /
# drwxrwxrwt 22 root root 4096 apr 29 19:07 tmp
```

Объяснить с 

`t` вместо `x` в блоке others – sticky bit установлен.

```
# Установить sticky bit
chmod +t /shared
chmod 1777 /shared    # 1 – это sticky bit в числовом виде
```

Объяснить с 

Сводная таблица



Слишком умный?

Пройди тест

владельца файла

SGID	2000	g+s	EGID = GID группы файла	Новые файлы наследуют группу директории
Sticky	1000	+t	Нет эффекта	Удалить файл может только его владелец

Специальные биты идут первой цифрой в четырёхзначной записи:

```
chmod 4755 file    # SUID + rwxr-xr-x
chmod 2755 dir     # SGID + rwxr-xr-x
chmod 1777 dir     # Sticky + rwxrwxrwx
chmod 3755 dir     # SGID + Sticky + rwxr-xr-x
```

Объяснить с 

9. ACL – когда стандартных прав недостаточно

Стандартная модель UGO имеет фундаментальное ограничение: один владелец и одна группа. Что если нужно дать доступ двум разным пользователям с разными правами? Например, пользователь maria должна читать и писать файл, пользователь sergei – только читать, а все остальные – ничего. В UGO это не реализовать. ACL (Access Control Lists) решают эту задачу – они позволяют прописать права для произвольного количества пользователей и групп на один объект.

Для начала проверим что пакет установлен:

```
apt list --installed 2>/dev/null | grep acl
# Если пакета нет:
sudo apt install acl
```

Объяснить с 

9.1 Просмотр ACL

```
getfacl file.txt
# file: file.txt
# owner: veil
# group: veil
user::rw-      ← права владельца (стандартные)
group::r--     ← права группы (стандартные)
other::r--     ← права остальных (стандартные)
```

Объяснить с 

Пока ACL не установлены – вывод совпадает с обычными правами. Как только добавляем ACL-запись, в выводе `ls -la` появляется символ `+` в конце строки прав:

```
ls -la file.txt
# -rw-r--r--+ 1 veil veil 34045 apr 29 15:18 file.txt
#           ↑ этот плюс означает что на файле есть ACL
```

Объяснить с 



Слишком умный?

Пройди тест

```
# Дать конкретному пользователю права на файл
setfacl -m u:maria:rw file.txt
```

```
# Дать группе права на директорию
setfacl -m g:backend:rx /var/www/api

# Несколько записей сразу
setfacl -m u:maria:rw,g:backend:r file.txt

# Удалить конкретную ACL-запись
setfacl -x u:maria file.txt

# Удалить все ACL с файла
setfacl -b file.txt
```

Объяснить с 

После добавления ACL для maria – `getfacl` покажет дополнительные строки:

```
getfacl file.txt
# file: file.txt
# owner: veil
# group: veil
user::rw-
user:maria:rw-  ← ACL-запись для конкретного пользователя
group::r--
mask::rw-      ← маска появилась автоматически
other::r--
```

Объяснить с 

9.3 Маска

Маска – это максимально возможные эффективные права для всех ACL-записей, кроме владельца и others. Если маска `r-x`, то даже ACL с `rwX` реально даст только `r-x`. Маска появляется автоматически при добавлении первой ACL-записи и расширяется когда вы добавляете записи с большими правами. Можно задать явно:

```
# Посмотреть маску
getfacl file.txt | grep mask

# Явно задать маску
setfacl -m mask::rx file.txt
```

Объяснить с 

9.4 ACL по умолчанию – важный момент

Без флага `-d` ACL применяется только к уже существующим файлам. Новые файлы созданные в директории его **не наследуют**. Это частая ошибка – настроил ACL на директорию, а файлы которые потом создаются внутри его не получают.

Правильный паттерн – всегда два вызова:

```
# 1. Применить к существующим файлам и поддиректориям
setfacl -R -m g:developers:rwX /project

# 2. Установить ACL по умолчанию – для всех новых файлов
setfacl -d -m u:ivan:rwX /project
```



Слишком умный?

Пройди тест

```
getfacl /project
# file: project
# owner: ivan
```

```
# group: ivan
user::rwx
group::r-x
other::r-x
default:group:developers:rwx ← будет применяться к новым файлам
```

[Объяснить с](#) 

10. umask – права по умолчанию при создании файлов

Когда создаётся новый файл или директория – система не даёт максимально возможные права, а вычитает из них маску. Это и есть `umask`. Максимальные права при создании: файл – `666` (execute никогда не ставится автоматически), директория – `777`.

```
# Посмотреть текущую маску
umask
# 0022

# Расчёт:
# файл:      666 - 022 = 644 (rw-r--r--)
# директория: 777 - 022 = 755 (rwxr-xr-x)
```

[Объяснить с](#) 

Строго говоря `umask` работает как побитовая операция исключения, а не простое вычитание. Разница проявляется если у вас нестандартные значения: например, `666` минус `033` математически даст `633`, но побитовое исключение даст `644`. Для стандартных значений (`022`, `027`, `077`) простое вычитание даёт верный результат и удобнее для понимания.

```
# Проверить на практике
umask 022
touch newfile.txt && ls -la newfile.txt
# -rw-r--r-- 1 ivan ivan 0 Mar 26 newfile.txt ← 644

mkdir newdir && ls -la | grep newdir
# drwxr-xr-x 2 ivan ivan 4096 Mar 26 newdir ← 755
```

[Объяснить с](#) 

Типичные значения

umask	Файлы	Директории	Когда применять
022	644	755	Стандарт – публичный доступ на чтение
027	640	750	Усиленный – others вообще без доступа
077	600	700	Параноидальный – только владелец

Где настраивается

```
# Для текущей сессии
umask 027
```

**Слишком умный?**

Пройди тест

UMASK 022

Для конкретного пользователя

```
echo "umask 027" >> ~/.bashrc
```

```
# Для сервиса через systemd unit-файл
# [Service]
# UMask=0027
```

[Объяснить с](#) 

Бонус и Заключение

Мы рассмотрели всё начиная от базовой модели UGO и битов `gwx` до алгоритма принятия решений ядром, специальных битов и ACL. Отдельно хочу поделиться полезным материалом на YouTube от Дмитрия с заданиями для самостоятельной отработки навыков, очень полезно тем, кто только начинает учить Linux.

- Урок 12 — Управление пользователями и группами
- Урок 13 — Избирательные права доступа (DAC) над файлами
- Урок 14 — Избирательные права доступа (DAC) над каталогами

Увидимся в следующей части серии!

Статья доступна на сайте [opensophy](#). Статьи, блоги и документации в первую очередь выходят и обновляются на сайте с указанием автора и соавторов.

Теги: linux, команды linux, linux для начинающих, linux для всех, command, terminal

Хабы: Учебный процесс в IT, Терминология IT, Инженерные системы, Информационная безопасность, Настройка Linux

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

[Подписаться](#)

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок

**16K+**

Охват за 30 дней

24

Карма

@opensophy

Пользователь

[Подписаться](#)**Слишком умный?**[Пройди тест](#)

Комментарии 8



- **opensophy**
13 мая в 12:22 
- Важное примечание: В этой статье намеренно остановились на «продвинутой базе». Тема прав доступа в контексте безопасности продолжится в следующих частях серии (и немного разберем их ещё в логи и мониторинг.)
-  Ответить  

**select26**
13 мая в 13:28 Ответить  **DS2**
13 мая в 13:51 Ответить  **opensophy**
14 мая в 10:16 Ответить  **denbog_php53**
13 мая в 17:53 Ответить  **feelamee**
14 мая в 05:12 

Слишком умный?

Пройди тест

Остановилось. выты group twx даже не рассматривались. twan является владельцем, но у владельца нет прав. Группа имеет полный доступ – но до неё проверка не дошла.

Решение – дать владельцу права явно:

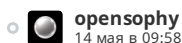
какое ж это решение...

В целом было бы хорошо объяснить зачем и почему и для чего нужны группы. В моем (чисто интуитивном) представлении это нужно чтобы давать какой-то группе пользователей определенные права - например доступ к файлам. Ведь намного проще добавить пользователя в группу/удалить из группы чем менять права везде.

Оказывается все не так просто... Было бы интересно узнать какое решение правильное. Либо для чего нужны группы тогда...



Ответить



opensophy
14 мая в 09:58



Здравствуйте! Да, вы правильно уловили проблему, пример получился скорее демонстрацией механики проверки прав ядром, чем реальной схемы использования групп.

Смысл того примера был показать, что права owner/group не суммируются: если процесс является владельцем файла, ядро использует только owner-биты и до group уже не доходит. Я уточню в статье отдельно, чтобы пример с ivan не создавал впечатление, будто группы «не работают как ожидается». Большое спасибо!



Ответить



Granulex
1 июн в 16:25

SUID на bash-скрипте ядро молча игнорирует – интерпретируемые файлы не получают setuid в Linux. Об этом узнают обычно после потраченного часа.



Ответить



Зарегистрируйтесь на Хабре, чтобы оставить комментарий

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



Nau515
22 часа назад

«Иностранные холодные кошельки» — или нами когда-нибудь перестанут управлять идиоты?

Простой

5 мин

18K

Мнение

+64

35

85



PatientZero
22 часа назад

Сэнди Петерсен и Джон Кармак: как Quake сломал id Software

3 мин

10K

Перевод



Слишком умный?

Пройди тест



Erwinmal
21 час назад

«Из ниоткуда в Грецию» и похищенный врач: что стояло за волной сообщений о странных дирижаблях в США 1890-х годов?

 Простой  14 мин  8.7K

Ретроспектива

 +23

 6

 4



faxenoff

7 часов назад

Taskbar с живыми превью работающих приложений (олдскул на NET11)

 Простой  3 мин  7.3K

Кейс

 +21

 14

 8



YMA

17 часов назад

Носители данных — «игра на выбывание» (2026.07)

 Простой  2 мин  8.4K

 +20

 5

 10



TrexSelectel

22 часа назад

5 новых одноплатников лета 2026 года: x86, Snapdragon и сменные процессоры

 Простой  7 мин  9.4K

 +19

 17

 3



Lunathecatt

17 часов назад

Самодельная электрогитара: тонируем деку и подгоняем к ней гриф

 Простой  8 мин  8.7K

Тutorial

 +16

 5

 3



k0mar0v

20 часов назад

Телефон из 1999 года: Alcatel One Touch Pocket. Что внутри?

 4 мин  9.3K

 +16

 3

 8



Small69

13 часов назад

Используем старый Mikrotik как сигнализацию



Слишком умный?

Пройди тест

 +15

 28

 7



BiktorSergeev

17 часов назад

Подземный гигант: зачем Швейцария строит самую мощную батарею в мире

🕒 4 мин

👁 9.2K

👍 +15

🔖 7

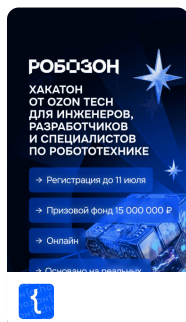
💬 32

Программируем дома: пишем о DIY-проектах, где пригодился код, и получаем призы

Турбо

Показать еще

ИСТОРИИ



Унеси с Робозона свои миллионы



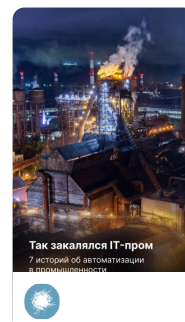
Годнота из блогов компаний



Из центра Млечного Пути



Пришёл, смастерил, рассказал — старт сезона DIY



Только IT-пром, только хардкор

ВАКАНСИИ

Senior DevOps-инженер

до 240 000 ₽ · AGIMA · Можно удаленно

PHP / Laravel разработчик

от 1 200 до 2 500 ₽ · ТАГА · Можно удаленно

Middle Fullstack Developer (PHP/Laravel + React)

от 2 000 до 2 700 \$ · Wiregate · Можно удаленно

Ведущий инженер Linux (г. Колпино)

до 130 000 ₽ · Гринатом · Колпино

Fullstack разработчик

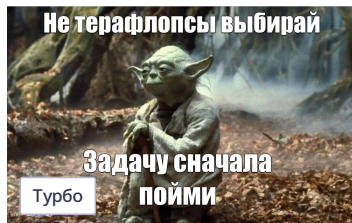
от 250 000 ₽ · Nevatech · Санкт-Петербург

Больше вакансий на Хабр Карьере

МИНУТОЧКУ ВНИМАНИЯ

**Слишком умный?**

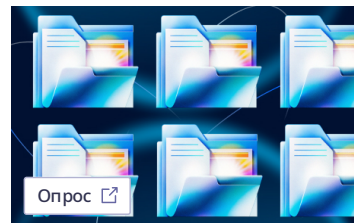
Пройди тест



Новое лучше, но есть нюанс:
выбираем GPU в 2026 году



Насколько глубоко ИИ проник в
разработку? Опрос



Расскажите, как вы ищете работу в
2026 году?

БЛИЖАЙШИЕ СОБЫТИЯ



2 марта – 10 августа

**PROBA Awards — международная коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

Маркетинг

Больше событий в календаре



Настройка языка

Техническая поддержка



Слишком умный?
Пройди тест